BITS WILP / NSP1

EvalSphere

Benchmark Smarter. Decide Faster. · Assignment 2

01 GITHUB REPOSITORY

Complete source code · requirements.txt · README.md · test CSV data

[PASTE YOUR GITHUB REPOSITORY URL HERE]

02 LIVE STREAMLIT COMMUNITY CLOUD APP

[PASTE YOUR STREAMLIT COMMUNITY CLOUD URL HERE]

03 BITS VIRTUAL LAB EXECUTION SCREENSHOT

ADD THE BITS LAB SCREENSHOT
Save as bits_virtual_lab_screenshot.png and regenerate the PDF.

Created by SRIRAM POLAMARASETTI

BITS ID 2026ak05608

Section 04 begins on the following page: complete project README content.

EvalSphere — Benchmark Smarter. Decide F

Created by: SRIRAM POLAMARASETTI BITS ID: 2026ak05608

EvalSphere is a unified workspace for model evaluation, benchmark analysis, performance insights, and dataset exploration.

a. Problem statement

Build, evaluate, compare, and demonstrate five machine-learning classification algorithms in an interactive Streamlit application. The project includes a general-purpose workbench for labeled tabular datasets and a separate, reproducible Mobile Price evaluation that satisfies the assignment dataset and saved-model checklist.

b. Dataset description

- Dataset: Mobile Price Classification
- Task: Multiclass classification; target price_range has labels 0, 1, 2,

and 3. These are price-range categories, not exact prices.

- Instances: 2,000
- Features: 20 numeric mobile specifications, including battery power,

clock speed, memory, camera specifications, screen dimensions, and connectivity.

- Class balance: 500 instances per class in the full dataset.
- Assignment threshold check: verified from dataset.csv — 2,000 records

(minimum 500) and 20 input features (minimum 12), with four classes and no missing values.

- Public source: Mobile Price Classification on Kaggle (https://www.kaggle.com/datasets/iabhishekofficial/mobile-price-classification)
- Pinned downloadable mirror: CSV source (https://raw.githubusercontent.com/coding-blocks-archives/machine-learning-online-2018/5c3cb8c3600ea1139833cc93c453c3b99f8a2688/Feature%20Selection/mobile-price-classification/train.csv)

The labeled source file is dataset.csv (./dataset.csv). Training uses one stratified 80/20 split with random seed 42. The saved scaler is fitted only on the training partition, preventing test data from influencing preprocessing. The generated test_data.csv (./test_data.csv) has 400 rows (100 per class). For quick app testing, sample_test_data.csv (./sample_test_data.csv) is a balanced 100-row subset (25 per class). Both CSVs contain the 20 features and the price_range target.

c. GitHub Repository Link

Repository URL: [PASTE YOUR GITHUB REPOSITORY URL HERE]

d. Models used and comparison

All five classifiers use the same split and standardized feature values. The SVM classifier uses sigmoid probability calibration so that it can report multiclass AUC as well as predictions. Precision, recall, and F1 use weighted averaging; AUC is weighted one-vs-rest. results.csv contains the exact full-precision values.

ML Model Name	Accuracy	AUC	Precision	Recall	F1	MCC
Logistic Regression	0.9650	0.9987	0.9650	0.9650	0.9650	0.9534
Decision Tree	0.8300	0.8867	0.8319	0.8300	0.8302	0.7738
KNN	0.5000	0.7697	0.5211	0.5000	0.5054	0.3350
Naive Bayes (Gaussian)	0.8100	0.9506	0.8113	0.8100	0.8105	0.7468
Support Vector Machine	0.8950	0.9580	0.8969	0.8950	0.8956	0.8602

Observations on model performance

ML Model Name	Observation about model performance
Logistic Regression	Observation about model performance: Best overall on this holdout: 96.50% accuracy and the highest MCC (0.9534), with excellent class ranking (AUC 0.9987).

Decision Tree	Observation about model performance: Reached 83.00% accuracy. Its lower AUC (0.8867) and MCC (0.7738) indicate weaker holdout performance than Logistic Regression and SVM.
KNN	Observation about model performance: Reached 50.00% accuracy, the lowest result in this comparison. With the current standardization and default neighborhood size of five, it is the weakest tested configuration.
Naive Bayes	Observation about model performance: Reached 81.00% accuracy. Its AUC (0.9506) is higher than the Decision Tree's despite lower ranking than Logistic Regression and SVM; the results are consistent with the models making different assumptions.
Support Vector Machine	Observation about model performance: Reached 89.50% accuracy and 0.8602 MCC, second only to Logistic Regression among these models on this split. Its calibrated AUC is 0.9580.

Overall winner: Logistic Regression, based on the highest holdout accuracy, F1, AUC, and MCC among the five evaluated models. This is a result on one reproducible holdout, not a guarantee of performance on every future dataset.

e. Streamlit application

- Live app URL: [PASTE YOUR STREAMLIT COMMUNITY CLOUD URL HERE]
- Student shown in UI: SRIRAM POLAMARASETTI · BITS ID 2026ak05608
- App experience: EvalSphere is a responsive, tab-driven product workspace

with Overview, Dataset Hub, Benchmark Lab, Feature Studio, Prediction Center, and Reports. It includes a custom orbital-sphere logo, light and dark themes, consistent Lucide-style SVG icons, keyboard focus states, reduced-motion support, and mobile layouts.

- Dataset gallery: One-click samples include Mobile Price, Titanic,

Heart Disease, Customer Churn, Breast Cancer, Diabetes, Bank Marketing, Employee Attrition, Iris, Wine Quality, and Credit Card Fraud. Mobile Price and the bundled scikit-learn Breast Cancer and Iris samples are local datasets. Other named samples are deterministic synthetic instructional examples, not real passenger, patient, customer, banking, or transaction records. The Wine Quality sample is generated for workflow practice and is not the public Wine Quality dataset. The gallery also includes the bundled Wine Classification dataset to preserve the original multiclass demo workflow.

- Any classification dataset: Upload a labeled CSV, select the target

column, choose a holdout percentage, and compare Logistic Regression, Decision Tree, K-Nearest Neighbors, Gaussian Naive Bayes, and SVM. Numeric, categorical, and mixed features are prepared inside leakage-safe pipelines. The comparison reports Accuracy, AUC, Precision, Recall, F1, MCC, and model training time and holdout prediction time. Choose the metric used to rank the five models. Ranked model cards include gold, silver, and bronze badges, progress bars, a metric leaderboard, performance matrix, and a champion summary. Search and filter the sample gallery by problem type and difficulty. Inspect class balance, feature types, training-partition feature relevance, numeric correlations, distributions, confidence-scored holdout predictions, confusion matrices, and per-class classification reports. Export predictions and benchmark reports as CSV or an executive PDF.

- Feature relevance: The generic benchmark estimates mutual information

using only the training partition. This is a general feature-target association view, not a causal explanation or a model-specific importance score.

- Timing: Generic training time covers fitting the preprocessing pipeline

and classifier. Prediction time covers the holdout prediction call and varies by machine. The saved assignment models are pre-trained, so their original training times are not recorded; their prediction timings are measured on the reference or uploaded test rows.

- Performance history: Results are shown for the current comparison only.

The application does not persist run history, so it avoids presenting misleading historical performance trends.

- Model playbook in the app: Logistic Regression is a quick linear

baseline; Decision Tree learns nonlinear rules; KNN uses distance and benefits from scaling; Gaussian Naive Bayes is a fast probabilistic baseline; and SVM can model complex boundaries but can take longer. The app describes these tradeoffs before users compare results.

- Sample data instructions: Search the Dataset Hub and load a sample

with one click, or upload your own labeled CSV. The app shows each sample's row count, feature count, problem type, difficulty, and source. In the Reports workspace, open the balanced 100-row sample for a quick metric preview or the complete 400-row holdout for the assignment evaluation. Upload either labeled CSV and select a model to see all six metrics, a confusion matrix, and a classification report. Download gallery datasets as CSV from the app for reuse.

- For this general-purpose mode, models are fitted on the uploaded dataset's

training split and evaluated on its stratified holdout. The test split is not used for fitting or preprocessing.

- Mobile Price assignment workflow: From Reports, open the saved-model

workflow to view the five models' comparison on the included 400-row holdout, download the full or balanced sample CSV, or upload a matching labeled test file to evaluate all five saved models without retraining. The selected model's metrics and per-class report are shown with its confusion matrix.

The interface includes a metric guide, per-model usage notes, dataset validation status, and quick-start datasets, so users can explore the benchmark before preparing their own file.

Data handling and responsive limits

- CSV uploads are limited to 20 MB. The general workbench supports 20–15,000 rows, at most 100 usable features, and at most 50 target classes; each categorical feature is limited to 1,000 distinct values. These limits prevent accidental resource exhaustion during five-model comparisons.
- The generic SVM uses a linear kernel with probability calibration to keep interactive runs more responsive. The saved assignment model uses its separately trained artifact.
- Uploaded data is processed in memory for the active session and is not written to project files or persisted by this application. Use only datasets you are authorized to upload. Do not upload executable/model files; user uploads are accepted as CSV data only.
- Saved joblib model files are executable serialization artifacts. Load only the model files bundled with this trusted project; never load joblib files supplied by an untrusted source.

The generic workbench expects a labeled classification CSV, with at least 20 rows and at least three examples per class. Binary and multiclass targets are supported. For binary AUC, the lexicographically last target label is treated as the positive class. Regression targets, unlabeled CSVs, and target columns with missing values are not supported.

Running locally

Use Python 3.11 or newer (the pinned scikit-learn version requires Python 3.11+):

```
cd Asiignment_2_ML
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt
python train_models.py
streamlit run app.py
```

python train_models.py regenerates both test CSVs, all six model/scaler artifacts, and results.csv. The reusable benchmark logic is in benchmark.py (./benchmark.py). The included notebook.ipynb (./notebook.ipynb) provides a compact dataset overview and comparison view after training. The scikit-learn version is pinned because the saved model artifacts are serialized with joblib and should be loaded with their training version. After replacing the URL placeholders and adding the BITS Virtual Lab screenshot as bits_virtual_lab_screenshot.png in the project folder, regenerate the PDF:

```
python prepare_submission_pdf.py
```

The script writes ML_Model_Comparator_Submission.pdf. It includes, in assignment order, the repository link and contents, live app link, BITS Lab screenshot area, and complete README content. The screenshot area remains an explicit placeholder until the screenshot is provided.

Repository contents

```
app.py
benchmark.py
train_models.py
prepare_submission_pdf.py
assets/
  evalsphere.svg
.streamlit/config.toml
requirements.txt
README.md
dataset.csv
test_data.csv
sample_test_data.csv
results.csv
notebook.ipynb
model/
  logistic_regression.pkl
  decision_tree.pkl
  knn.pkl
  naive_bayes.pkl
  svm.pkl
  scaler.pkl
```

Final submission checklist

- [x] Source code, README, requirements, original dataset, sample and full test

data, comparison metrics, and saved model/preprocessing artifacts are in the project.

- [ ] GitHub: Push the complete project and replace the repository placeholder in section c with a clickable link.

- [ ] Streamlit Community Cloud: Deploy app.py and replace the live app placeholder in section e with a clickable link.
- [ ] BITS Virtual Lab: Run the assignment on the lab and capture the required execution screenshot.
- [ ] Single PDF: Replace both URL placeholders, add bits_virtual_lab_screenshot.png, regenerate the PDF, and confirm it contains the GitHub link and repository contents, live app link, BITS Lab screenshot, and complete README in that order.

GitHub publishing, Cloud deployment, and the BITS Lab screenshot require the student's accounts and lab session; they are not represented as completed here.